

Hosting

Version Control Systems

We already saw how Python allows you to install packages from version control systems like GitHub. Python also supports Mercurial, Subversion, and Bazaar. We are going to talk about how a package would be pushed to GitHub. **Note**, we are not going to push the `shapes` package to GitHub.

The first thing you need to do is create a GitHub account. Then you need to create a repository for the package. If we were to push the `shapes-package` directory to GitHub, it would include the directory for the virtual environment. Virtual environments are great for local Python development. However, they are not required for the package once it has been created.

GitHub allows you to create a `.gitignore` file. This is a plain text file which tells git to not upload certain files and directories. First create the `.gitignore` file. Then add the following line of code to ignore the `shapes-env` directory which is the virtual environment:

▼ **Did you notice?**
There is a leading dot (`.`) for the `.gitignore` file. It is required.

```
/shapes-env
```

Once you are ignoring the virtual environment, you can follow the normal git workflow. Adding a license and `README` file would be helpful for people looking to use your package. See the documentation for more information on how to use the git.

PyPI

Preparing your package for use on PyPI is much more complex than the simple wheel creation from the previous page. The setup file provides the metadata, which is still required. You also need to include a license, a `.toml` file that tells Python which build tools to use. The `README` file gives users useful information on how to use the package. You are encouraged to have tests for your packages. If you do not have any tests, create the directory but leave it empty.

```
packaging_tutorial/
├── LICENSE
├── pyproject.toml
├── README.md
├── setup.cfg
├── src/
│   └── example_package/
│       ├── __init__.py
│       └── example.py
└── tests/
```

The process does not end here. You need to upload your package to TestPyPI. TestPyPI requires an account and a token. Once your package has passed all of your testing, it is ready for PyPI itself.

Reading Question

True or False: Once you build your wheel, it is ready for upload to a hosting site without any other work that needs to be done.

- ☐ True
- ☒ False

False. Uploading your package to a public site for distribution requires some extra work. How much depends on where you want to upload it. Choosing GitHub over PyPI reduces the amount of work. However, you will still need a `.gitignore` to ignore the virtual environment. Having files for a `README` and license would also be helpful.

Check It!

Next